

AI/ML Architecture & Implementation

Overview

The AI/ML Architecture forms the intelligent core of the sales-enablement platform, implementing sophisticated machine learning systems with Llama 3.1, LoRA fine-tuning, advanced RAG capabilities, and enterprise-grade model serving infrastructure. This document details the comprehensive AI/ML implementation strategy, hosting options, data flow architecture, and scalability patterns.

AI/ML System Architecture

Core AI/ML Framework

```
class AIMLFramework:
    def __init__(self):
        self.model_manager = ModelManager()
        self.inference_engine = InferenceEngine()
        self.training_orchestrator = TrainingOrchestrator()
        self.rag_engine = RAGEngine()
        self.feature_store = FeatureStore()
        self.model_registry = ModelRegistry()
        self.serving_infrastructure = ServingInfrastructure()

    async def initialize(self):
        # Initialize all AI/ML components
        await self.model_manager.initialize()
        await self.inference_engine.start()
        await self.training_orchestrator.start()
        await self.rag_engine.initialize()
        await self.feature_store.connect()
        await self.serving_infrastructure.deploy()
```

1. LLM Hosting Strategies

Self-Hosted vs Cloud Deployment

Hybrid Hosting Architecture:

```
class LLMHostingManager:
    def __init__(self):
        self.self_hosted_cluster = SelfHostedCluster()
        self.cloud_endpoints = CloudEndpoints()
        self.load_balancer = IntelligentLoadBalancer()
        self.cost_optimizer = CostOptimizer()

    async def determine_optimal_hosting(self, request_context):
        # Analyze request characteristics
        analysis = await self.analyze_request(request_context)

        # Determine optimal hosting strategy
        if analysis.data_sensitivity == "high":
            return await self.route_to_self_hosted(request_context)
        elif analysis.latency_requirement == "ultra_low":
            return await self.route_to_edge_deployment(request_context)
        elif analysis.cost_sensitivity == "high":
            return await self.route_to_cloud_spot_instances(request_context)
        else:
            return await self.route_to_optimal_endpoint(request_context)
```

Self-Hosted Infrastructure

On-Premises LLM Cluster:

```

class SelfHostedLLMCluster:
    def __init__(self):
        self.gpu_cluster = GPUCluster()
        self.model_sharding = ModelSharding()
        self.tensor_parallel = TensorParallelism()
        self.pipeline_parallel = PipelineParallelism()
        self.memory_optimizer = MemoryOptimizer()

    async def deploy_llama_cluster(self, model_config):
        # Calculate optimal sharding strategy
        sharding_strategy = await self.calculate_sharding_strategy(
            model_size=model_config.parameter_count,
            available_gpus=self.gpu_cluster.available_gpus,
            memory_per_gpu=self.gpu_cluster.memory_per_gpu
        )

        # Deploy model with optimal parallelization
        deployment_config = ModelDeploymentConfig(
            model_path=model_config.model_path,
            tensor_parallel_size=sharding_strategy.tensor_parallel_size,
            pipeline_parallel_size=sharding_strategy.pipeline_parallel_size,
            max_batch_size=sharding_strategy.optimal_batch_size,
            max_sequence_length=model_config.max_sequence_length,
            quantization=model_config.quantization_config
        )

        # Initialize model servers
        model_servers = []
        for gpu_group in sharding_strategy.gpu_groups:
            server = await self.create_model_server(
                gpu_group=gpu_group,
                deployment_config=deployment_config
            )
            model_servers.append(server)

        # Set up load balancing
        load_balancer = await self.setup_load_balancer(model_servers)

        return LLMClusterDeployment(
            model_servers=model_servers,
            load_balancer=load_balancer,
            sharding_strategy=sharding_strategy,
            deployment_config=deployment_config
        )

```

GPU Resource Management:

```

class GPUResourceManager:
    def __init__(self):
        self.gpu_monitor = GPUMonitor()
        self.memory_manager = GPUMemoryManager()
        self.scheduler = GPUScheduler()

    async def optimize_gpu_utilization(self):
        # Monitor GPU usage across cluster
        gpu_stats = await self.gpu_monitor.get_cluster_stats()

        # Identify optimization opportunities
        optimizations = []

        for gpu_id, stats in gpu_stats.items():
            if stats.memory_utilization < 0.7:
                # GPU has available memory - can handle more load
                optimizations.append(
                    GPUOptimization(
                        gpu_id=gpu_id,
                        type="increase_batch_size",
                        current_utilization=stats.memory_utilization,
                        recommended_action="Increase batch size by 20%"
                    )
                )
            elif stats.compute_utilization < 0.8:
                # GPU compute is underutilized
                optimizations.append(
                    GPUOptimization(
                        gpu_id=gpu_id,
                        type="optimize_kernels",
                        current_utilization=stats.compute_utilization,
                        recommended_action="Apply kernel fusion optimizations"
                    )
                )

        # Apply optimizations
        for optimization in optimizations:
            await self.apply_optimization(optimization)

        return optimizations

```

Cloud Hosting Strategy

Multi-Cloud LLM Deployment:

```

class CloudLLMDeployment:
    def __init__(self):
        self.aws_deployment = AWSLLMDeployment()
        self.azure_deployment = AzureLLMDeployment()
        self.gcp_deployment = GCPLLMDeployment()
        self.cost_analyzer = CloudCostAnalyzer()

    async def deploy_multi_cloud(self, deployment_requirements):
        # Analyze cost and performance across clouds
        cloud_analysis = await self.analyze_cloud_options(deployment_requirements)

        # Deploy to optimal cloud providers
        deployments = {}

        if cloud_analysis.aws_optimal_for:
            aws_deployment = await self.aws_deployment.deploy(
                model_config=deployment_requirements.model_config,
                instance_types=cloud_analysis.aws_optimal_instances,
                regions=cloud_analysis.aws_optimal_regions
            )
            deployments["aws"] = aws_deployment

        if cloud_analysis.azure_optimal_for:
            azure_deployment = await self.azure_deployment.deploy(
                model_config=deployment_requirements.model_config,
                vm_sizes=cloud_analysis.azure_optimal_vms,
                regions=cloud_analysis.azure_optimal_regions
            )
            deployments["azure"] = azure_deployment

        # Set up cross-cloud load balancing
        global_load_balancer = await self.setup_global_load_balancer(deployments)

        return MultiCloudDeployment(
            deployments=deployments,
            global_load_balancer=global_load_balancer,
            cost_optimization=cloud_analysis.cost_optimization
        )

```

2. Advanced Model Serving Infrastructure

High-Performance Inference Engine

vLLM Integration:

```

class vLLMInferenceEngine:
    def __init__(self):
        self.vllm_engine = None
        self.sampling_params = SamplingParams()
        self.request_queue = AsyncQueue()
        self.batch_processor = BatchProcessor()

    async def initialize(self, model_config):
        # Initialize vLLM engine with optimizations
        self.vllm_engine = AsyncLLMEngine.from_engine_args(
            EngineArgs(
                model=model_config.model_path,
                tensor_parallel_size=model_config.tensor_parallel_size,
                dtype=model_config.dtype,
                max_model_len=model_config.max_sequence_length,
                gpu_memory_utilization=0.9,
                swap_space=4, # 4GB swap space
                enforce_eager=False, # Enable CUDA graphs
                max_context_len_to_capture=8192,
                disable_log_stats=False
            )
        )

        # Start request processing
        asyncio.create_task(self.process_requests())

    async def generate(self, prompt, generation_config):
        # Create sampling parameters
        sampling_params = SamplingParams(
            temperature=generation_config.temperature,
            top_p=generation_config.top_p,
            top_k=generation_config.top_k,
            max_tokens=generation_config.max_tokens,
            stop=generation_config.stop_sequences,
            frequency_penalty=generation_config.frequency_penalty,
            presence_penalty=generation_config.presence_penalty
        )

        # Add request to queue
        request_id = str(uuid.uuid4())
        request = InferenceRequest(
            request_id=request_id,
            prompt=prompt,
            sampling_params=sampling_params,
            timestamp=datetime.utcnow()
        )

        await self.request_queue.put(request)

        # Wait for result
        result = await self.wait_for_result(request_id)
        return result

    async def process_requests(self):
        while True:
            # Collect batch of requests
            batch = await self.collect_batch()

            if batch:
                # Process batch
                await self.process_batch(batch)

```

```

        await asyncio.sleep(0.001) # Small delay to prevent busy waiting

    async def process_batch(self, batch):
        # Prepare batch for vLLM
        prompts = [req.prompt for req in batch]
        sampling_params_list = [req.sampling_params for req in batch]
        request_ids = [req.request_id for req in batch]

        # Generate responses
        results = await self.vllm_engine.generate(
            prompts=prompts,
            sampling_params=sampling_params_list,
            request_ids=request_ids
        )

        # Process results
        for result in results:
            await self.handle_result(result)

```

TensorRT-LLM Integration:

```

class TensorRTLLMEngine:
    def __init__(self):
        self.trt_engine = None
        self.tokenizer = None
        self.runtime = None

    async def initialize(self, model_config):
        # Load TensorRT engine
        self.trt_engine = await self.load_tensorrt_engine(
            model_config.tensorrt_engine_path
        )

        # Initialize tokenizer
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_config.tokenizer_path
        )

        # Create runtime
        self.runtime = TensorRTLLMRuntime(
            engine=self.trt_engine,
            tokenizer=self.tokenizer
        )

    async def generate(self, prompt, generation_config):
        # Tokenize input
        input_ids = self.tokenizer.encode(prompt, return_tensors="pt")

        # Generate with TensorRT-LLM
        output_ids = await self.runtime.generate(
            input_ids=input_ids,
            max_new_tokens=generation_config.max_tokens,
            temperature=generation_config.temperature,
            top_p=generation_config.top_p,
            do_sample=generation_config.do_sample,
            pad_token_id=self.tokenizer.pad_token_id,
            eos_token_id=self.tokenizer.eos_token_id
        )

        # Decode output
        generated_text = self.tokenizer.decode(
            output_ids[0][len(input_ids[0]):],
            skip_special_tokens=True
        )

        return GenerationResult(
            generated_text=generated_text,
            input_length=len(input_ids[0]),
            output_length=len(output_ids[0]) - len(input_ids[0]),
            generation_time=time.time() - start_time
        )

```

Dynamic Model Loading and Scaling

Model Hot-Swapping System:

```

class ModelHotSwapper:
    def __init__(self):
        self.active_models = {}
        self.model_cache = ModelCache()
        self.traffic_router = TrafficRouter()

    async def swap_model(self, model_id, new_model_config):
        # Load new model
        new_model = await self.load_model(new_model_config)

        # Validate new model
        validation_result = await self.validate_model(new_model)
        if not validation_result.is_valid:
            raise ModelValidationError(validation_result.errors)

        # Gradual traffic migration
        migration_plan = await self.create_migration_plan(
            old_model_id=model_id,
            new_model=new_model,
            migration_strategy="canary" # or "blue_green"
        )

        # Execute migration
        migration_result = await self.execute_migration(migration_plan)

        # Update active models
        if migration_result.success:
            old_model = self.active_models.get(model_id)
            self.active_models[model_id] = new_model

            # Cleanup old model
            if old_model:
                await self.cleanup_model(old_model)

        return migration_result

    async def execute_migration(self, migration_plan):
        if migration_plan.strategy == "canary":
            return await self.execute_canary_migration(migration_plan)
        elif migration_plan.strategy == "blue_green":
            return await self.execute_blue_green_migration(migration_plan)
        else:
            raise ValueError(f"Unknown migration strategy: {migration_plan.strategy}")

    async def execute_canary_migration(self, migration_plan):
        # Start with 5% traffic to new model
        await self.traffic_router.update_routing(
            model_id=migration_plan.model_id,
            old_model_weight=0.95,
            new_model_weight=0.05
        )

        # Monitor performance for 10 minutes
        performance_metrics = await self.monitor_performance(
            duration=timedelta(minutes=10)
        )

        if performance_metrics.meets_criteria():
            # Gradually increase traffic to new model
            traffic_steps = [0.1, 0.25, 0.5, 0.75, 1.0]

            for new_weight in traffic_steps:

```

```

        await self.traffic_router.update_routing(
            model_id=migration_plan.model_id,
            old_model_weight=1.0 - new_weight,
            new_model_weight=new_weight
        )

        # Monitor each step
        step_metrics = await self.monitor_performance(
            duration=timedelta(minutes=5)
        )

        if not step_metrics.meets_criteria():
            # Rollback
            await self.rollback_migration(migration_plan)
            return MigrationResult(success=False, reason="Performance degrada-
tion")

        return MigrationResult(success=True)

```

3. Advanced RAG Implementation

Sophisticated RAG Engine

Multi-Modal RAG System:

```

class AdvancedRAGEngine:
    def __init__(self):
        self.vector_store = VectorStore()
        self.knowledge_graph = KnowledgeGraph()
        self.embedding_models = EmbeddingModels()
        self.retrieval_optimizer = RetrievalOptimizer()
        self.context_ranker = ContextRanker()

    async def retrieve_and_rank(self, query, context):
        # Multi-modal retrieval
        retrieval_tasks = [
            self.vector_retrieval(query, context),
            self.graph_retrieval(query, context),
            self.hybrid_retrieval(query, context)
        ]

        retrieval_results = await asyncio.gather(*retrieval_tasks)

        # Combine and rank results
        combined_results = await self.combine_retrieval_results(retrieval_results)
        ranked_results = await self.context_ranker.rank(combined_results, query)

        # Optimize context window
        optimized_context = await self.optimize_context_window(
            ranked_results, context.max_tokens
        )

        return RAGResult(
            retrieved_documents=ranked_results,
            optimized_context=optimized_context,
            retrieval_metadata=self.generate_retrieval_metadata(retrieval_results)
        )

    async def vector_retrieval(self, query, context):
        # Generate query embedding
        query_embedding = await self.embedding_models.embed_query(query)

        # Semantic search
        similar_docs = await self.vector_store.similarity_search(
            query_embedding=query_embedding,
            top_k=context.retrieval_top_k,
            filters=context.filters,
            score_threshold=context.similarity_threshold
        )

        return VectorRetrievalResult(
            documents=similar_docs,
            retrieval_method="vector_similarity",
            scores=[doc.similarity_score for doc in similar_docs]
        )

    async def graph_retrieval(self, query, context):
        # Extract entities from query
        entities = await self.extract_entities(query)

        # Graph traversal for related information
        related_nodes = []
        for entity in entities:
            nodes = await self.knowledge_graph.find_related_nodes(
                entity=entity,
                max_hops=context.max_graph_hops,
                relationship_types=context.allowed_relationships
            )

```

```
    )
    related_nodes.extend(nodes)

    # Convert graph nodes to documents
    documents = await self.nodes_to_documents(related_nodes)

    return GraphRetrievalResult(
        documents=documents,
        retrieval_method="knowledge_graph",
        entities=entities,
        graph_paths=self.extract_graph_paths(related_nodes)
    )
```

Intelligent Context Ranking:

```

class ContextRanker:
    def __init__(self):
        self.relevance_model = RelevanceModel()
        self.diversity_calculator = DiversityCalculator()
        self.freshness_scorer = FreshnessScorer()

    async def rank(self, documents, query):
        # Calculate multiple ranking signals
        ranking_signals = await asyncio.gather(
            self.calculate_relevance_scores(documents, query),
            self.calculate_diversity_scores(documents),
            self.calculate_freshness_scores(documents),
            self.calculate_authority_scores(documents)
        )

        relevance_scores = ranking_signals[0]
        diversity_scores = ranking_signals[1]
        freshness_scores = ranking_signals[2]
        authority_scores = ranking_signals[3]

        # Combine scores with learned weights
        final_scores = []
        for i, doc in enumerate(documents):
            combined_score = (
                0.5 * relevance_scores[i] +
                0.2 * diversity_scores[i] +
                0.2 * freshness_scores[i] +
                0.1 * authority_scores[i]
            )
            final_scores.append(combined_score)

        # Sort by combined score
        ranked_indices = sorted(
            range(len(documents)),
            key=lambda i: final_scores[i],
            reverse=True
        )

        ranked_documents = [documents[i] for i in ranked_indices]

        return RankedDocuments(
            documents=ranked_documents,
            scores=final_scores,
            ranking_metadata={
                "relevance_scores": relevance_scores,
                "diversity_scores": diversity_scores,
                "freshness_scores": freshness_scores,
                "authority_scores": authority_scores
            }
        )

```

Dynamic Knowledge Base Updates

Real-Time Knowledge Ingestion:

```

class KnowledgeIngestionPipeline:
    def __init__(self):
        self.document_processor = DocumentProcessor()
        self.embedding_generator = EmbeddingGenerator()
        self.knowledge_extractor = KnowledgeExtractor()
        self.vector_updater = VectorUpdater()
        self.graph_updater = GraphUpdater()

    async def ingest_document(self, document):
        # Process document
        processed_doc = await self.document_processor.process(document)

        # Extract knowledge
        knowledge = await self.knowledge_extractor.extract(processed_doc)

        # Generate embeddings
        embeddings = await self.embedding_generator.generate(processed_doc)

        # Update vector store
        await self.vector_updater.add_document(
            document=processed_doc,
            embeddings=embeddings
        )

        # Update knowledge graph
        await self.graph_updater.add_knowledge(knowledge)

        # Trigger index optimization if needed
        if await self.should_optimize_indices():
            await self.optimize_indices()

        return IngestionResult(
            document_id=processed_doc.id,
            embeddings_generated=len(embeddings),
            entities_extracted=len(knowledge.entities),
            relationships_added=len(knowledge.relationships)
        )

    async def optimize_indices(self):
        # Optimize vector indices
        await self.vector_store.optimize_indices()

        # Optimize graph indices
        await self.knowledge_graph.optimize_indices()

        # Update retrieval statistics
        await self.update_retrieval_statistics()

```

4. Feature Store and Data Pipeline

Enterprise Feature Store

Comprehensive Feature Store:

```

class FeatureStore:
    def __init__(self):
        self.online_store = OnlineFeatureStore() # Redis/DynamoDB
        self.offline_store = OfflineFeatureStore() # S3/BigQuery
        self.feature_registry = FeatureRegistry()
        self.feature_pipeline = FeaturePipeline()

    async def get_features(self, entity_id, feature_names, timestamp=None):
        # Get features from online store for real-time inference
        if timestamp is None:
            features = await self.online_store.get_features(
                entity_id=entity_id,
                feature_names=feature_names
            )
        else:
            # Point-in-time lookup for training data
            features = await self.offline_store.get_features_at_time(
                entity_id=entity_id,
                feature_names=feature_names,
                timestamp=timestamp
            )

        return features

    async def register_feature_group(self, feature_group_config):
        # Register new feature group
        feature_group = FeatureGroup(
            name=feature_group_config.name,
            entity_type=feature_group_config.entity_type,
            features=feature_group_config.features,
            source=feature_group_config.source,
            transformation=feature_group_config.transformation,
            schedule=feature_group_config.schedule
        )

        await self.feature_registry.register(feature_group)

        # Set up feature pipeline
        pipeline = await self.feature_pipeline.create_pipeline(feature_group)
        await pipeline.start()

        return feature_group

```

Real-Time Feature Pipeline:

```

class RealTimeFeaturePipeline:
    def __init__(self):
        self.stream_processor = StreamProcessor()
        self.feature_transformer = FeatureTransformer()
        self.feature_validator = FeatureValidator()

    async def process_feature_stream(self, feature_group):
        # Set up stream processing
        stream = await self.stream_processor.create_stream(
            source=feature_group.source,
            processing_config=feature_group.processing_config
        )

        async for event in stream:
            try:
                # Transform raw data to features
                features = await self.feature_transformer.transform(
                    raw_data=event.data,
                    transformation_config=feature_group.transformation
                )

                # Validate features
                validation_result = await self.feature_validator.validate(
                    features=features,
                    schema=feature_group.schema
                )

                if validation_result.is_valid:
                    # Store in online feature store
                    await self.online_store.store_features(
                        entity_id=event.entity_id,
                        features=features,
                        timestamp=event.timestamp
                    )
                else:
                    # Log validation errors
                    await self.log_validation_errors(
                        event, validation_result.errors
                    )

            except Exception as e:
                await self.handle_processing_error(event, e)

```

5. Model Lifecycle Management

Comprehensive Model Registry

MLflow-Based Model Registry:

```

class ModelRegistry:
    def __init__(self):
        self.mlflow_client = MlflowClient()
        self.model_store = ModelStore()
        self.version_manager = VersionManager()
        self.deployment_manager = DeploymentManager()

    async def register_model(self, model_info, artifacts):
        # Register model with MLflow
        model_version = await self.mlflow_client.create_model_version(
            name=model_info.name,
            source=artifacts.model_path,
            description=model_info.description,
            tags=model_info.tags
        )

        # Store model artifacts
        await self.model_store.store_artifacts(
            model_version=model_version,
            artifacts=artifacts
        )

        # Create version metadata
        version_metadata = ModelVersionMetadata(
            version=model_version.version,
            model_config=model_info.config,
            training_metadata=artifacts.training_metadata,
            evaluation_metrics=artifacts.evaluation_metrics,
            dependencies=artifacts.dependencies,
            created_at=datetime.utcnow()
        )

        await self.version_manager.store_metadata(
            model_version, version_metadata
        )

        return model_version

    async def promote_model(self, model_name, version, stage):
        # Validate promotion criteria
        validation_result = await self.validate_promotion(
            model_name, version, stage
        )

        if not validation_result.is_valid:
            raise ModelPromotionError(validation_result.errors)

        # Update model stage
        await self.mlflow_client.transition_model_version_stage(
            name=model_name,
            version=version,
            stage=stage
        )

        # Trigger deployment if promoting to production
        if stage == "Production":
            await self.deployment_manager.deploy_to_production(
                model_name, version
            )

        return PromotionResult(
            model_name=model_name,

```

```

        version=version,
        new_stage=stage,
        promoted_at=datetime.utcnow()
    )

```

Automated Model Deployment

CI/CD for ML Models:

```

class ModelDeploymentPipeline:
    def __init__(self):
        self.build_system = ModelBuildSystem()
        self.test_runner = ModelTestRunner()
        self.deployment_orchestrator = DeploymentOrchestrator()
        self.monitoring_setup = MonitoringSetup()

    async def deploy_model(self, model_config, deployment_config):
        # Build model artifacts
        build_result = await self.build_system.build_model(model_config)

        # Run model tests
        test_result = await self.test_runner.run_tests(
            model_artifacts=build_result.artifacts,
            test_config=deployment_config.test_config
        )

        if not test_result.all_passed:
            raise ModelTestFailureError(test_result.failed_tests)

        # Deploy to staging
        staging_deployment = await self.deployment_orchestrator.deploy_to_staging(
            model_artifacts=build_result.artifacts,
            deployment_config=deployment_config
        )

        # Run integration tests
        integration_result = await self.test_runner.run_integration_tests(
            staging_deployment
        )

        if integration_result.all_passed:
            # Deploy to production
            production_deployment = await self.deployment_orchestrator.de-
            ploy_to_production(
                model_artifacts=build_result.artifacts,
                deployment_config=deployment_config
            )

            # Set up monitoring
            await self.monitoring_setup.setup_model_monitoring(
                production_deployment
            )

            return DeploymentResult(
                staging_deployment=staging_deployment,
                production_deployment=production_deployment,
                monitoring_config=self.monitoring_setup.config
            )
        else:
            raise IntegrationTestFailureError(integration_result.failed_tests)

```

6. Scalability and Reliability Patterns

Auto-Scaling Infrastructure

Intelligent Auto-Scaler:

```

class IntelligentAutoScaler:
    def __init__(self):
        self.metrics_collector = MetricsCollector()
        self.predictor = LoadPredictor()
        self.scaler = ResourceScaler()

    async def auto_scale(self):
        while True:
            # Collect current metrics
            current_metrics = await self.metrics_collector.get_current_metrics()

            # Predict future load
            predicted_load = await self.predictor.predict_load(
                current_metrics=current_metrics,
                prediction_horizon=timedelta(minutes=15)
            )

            # Determine scaling action
            scaling_decision = await self.determine_scaling_action(
                current_metrics, predicted_load
            )

            if scaling_decision.action != "no_action":
                await self.scaler.execute_scaling(scaling_decision)

            await asyncio.sleep(60) # Check every minute

    async def determine_scaling_action(self, current_metrics, predicted_load):
        # Calculate resource utilization
        cpu_utilization = current_metrics.cpu_utilization
        memory_utilization = current_metrics.memory_utilization
        gpu_utilization = current_metrics.gpu_utilization
        queue_length = current_metrics.request_queue_length

        # Predict future utilization
        predicted_cpu = predicted_load.cpu_utilization
        predicted_memory = predicted_load.memory_utilization
        predicted_gpu = predicted_load.gpu_utilization

        # Scaling thresholds
        scale_up_threshold = 0.8
        scale_down_threshold = 0.3

        if (predicted_cpu > scale_up_threshold or
            predicted_memory > scale_up_threshold or
            predicted_gpu > scale_up_threshold or
            queue_length > 100):

            return ScalingDecision(
                action="scale_up",
                target_replicas=self.calculate_target_replicas(predicted_load),
                reason="High predicted utilization or queue length"
            )

        elif (cpu_utilization < scale_down_threshold and
              memory_utilization < scale_down_threshold and
              gpu_utilization < scale_down_threshold and
              queue_length < 10):

            return ScalingDecision(
                action="scale_down",
                target_replicas=max(1, current_metrics.current_replicas - 1),

```

```
        reason="Low utilization across all metrics"  
    )  
  
    return ScalingDecision(action="no_action")
```

This comprehensive AI/ML architecture provides enterprise-grade machine learning capabilities with sophisticated model serving, advanced RAG implementation, and robust scalability patterns, ensuring optimal performance and reliability for the AI sales-enablement platform.